Индексы: B-tree, hash и bitmap

Table of contents

LSM-дерево: индексирование под запись . . . 2
- Строительные блоки . . . 2
  - SSTable . . . 2
  - Memtable . . . 3
  - Bloom Filter . . . 4
  - WAL . . . 8
- Полная структура . . . 8
- Пайплайн в движении . . . 9
- Сложность операций . . . 9
  - Create / Update . . . 9
  - Read . . . 9
  - Delete . . . 10
  - Range Query . . . 10
- Сравнение с B-tree и Hash . . . 10
  - LSM vs B-tree . . . 10
  - LSM vs Hash . . . 10
- Практическое применение . . . 10

B-дерево: индексирование на диске . . . 11
- Почему для диска используют B-дерево . . . 11
- Изменяемые страницы против иммутабельных файлов . . . 11
- Структура и коэффициент ветвления . . . 12
  - Инварианты . . . 13
  - Пример узла при `B = 4` . . . 13
- Вставка и балансировка . . . 13
- Сложность операций . . . 15
  - Вставка . . . 15
  - Поиск . . . 15
  - Обновление . . . 15
  - Удаление . . . 15
  - Диапазонный запрос . . . 15
- Сравнение с LSM-деревом по I/O . . . 15
  - B-дерево и LSM-дерево . . . 15
- Практическое применение . . . 16

Индексы: B-tree, hash и bitmap . . . 16
- Что такое индекс . . . 16
- Почему индекс ускоряет чтение, но замедляет запись . . . 17
- B-tree как индекс по умолчанию . . . 17

Hash-индекс 17
Bitmap-индекс 18
Как выбирать индекс 18
Связь с двумя предыдущими лекциями 19

Сборная страница со всеми лекциями курса AIS. Доступна версия для печати (PDF).

Отдельные лекции по прямым ссылкам:

- LSM-дерево: индексирование под запись
- B-дерево: индексирование на диске
- Индексы: B-tree, hash и bitmap

LSM-дерево: индексирование под запись

Первая структура, которую мы изучаем, это **LSM-дерево** (Log-Structured Merge Tree). LSM оптимизировано под **высокую скорость записи**. Это критично для систем, где запись доминирует над чтением: логи, метрики, IoT-телеметрия, временны́е ряды.

Идея в трёх шагах:

1. Записи сначала попадают в память в структуру **Memtable**.
2. При заполнении Memtable замораживается и сбрасывается на диск как отсортированный файл **SSTable**.
3. Периодически файлы сливаются фоновой **компакцией**, чтобы избежать фрагментации и удалить устаревшие версии.

LSM используется в Cassandra, RocksDB, LevelDB, HBase, Google Bigtable. Компромисс понятен: быстрая запись, но чтение может потребовать проверки нескольких файлов (read amplification).

Строительные блоки

SSTable

SSTable (Sorted String Table) это иммутабельный файл с парами `(ключ, значение)`, отсортированными по ключу.

Идея: данные пишутся крупными отсортированными блоками, что делает последовательное чтение быстрым и удобным для диапазонных запросов.

Что даёт:

- дешёвая запись на диск большими порциями;
- быстрые диапазонные чтения благодаря сортировке;
- предсказуемый I/O за счёт последовательного доступа.

Что внутри:

- набор отсортированных блоков данных;

- лёгкий индекс на блоки, чтобы не читать весь файл.

Go-псевдоинтерфейс и оценки:

```go
// SSTable: иммутабельное чтение по ключу и диапазону.
type SSTable interface {
    // Get: бинарный поиск по индексу + чтение блока.
    // O(log b + k), b — число блоков, k — размер блока.
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: последовательное чтение по диапазону.
    // O(log b + r), r — число записей в диапазоне.
    RangeScan(start, end []byte, fn func(k, v []byte))
}
```

Memtable

Memtable это изменяемая структура в памяти для накопления записей перед сбросом в SSTable.

Смысл: быстрые записи в память, а на диск данные уходят порциями уже отсортированными.

Что внутри:

- упорядоченная структура в памяти, обычно skip list или RB-дерево;
- быстрая поддержка вставки и поиска по ключу.

Go-псевдоинтерфейс и оценки:

```go
// Memtable: изменяемые записи, сортировка в памяти.
type Memtable interface {
    // Put: вставка или обновление.
    // O(log m), m — число ключей.
    Put(key, value []byte)

    // Get: поиск по ключу.
    // O(log m).
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: диапазонный проход.
    // O(log m + r).
    RangeScan(start, end []byte, fn func(k, v []byte))

    // Flush: упорядоченный сброс в новый SSTable.
    // O(m) на последовательную запись.
```

```
    Flush(builder SSTableBuilder)
}
```

Bloom Filter

Bloom Filter это компактная вероятностная структура, которая отвечает на один вопрос: «может ли ключ x лежать в этом множестве?». Ответа два: **точно нет** или **возможно, да**. Ложноположительные срабатывания возможны, ложноотрицательных не бывает никогда.

Для LSM это точно то, что нужно: у нас десятки SSTable на диске, и перед чтением каждого хочется быстро отсечь те, где искомого ключа заведомо нет, чтобы не тратить дисковые обращения.

Как устроен

Внутри фильтра всего две вещи:

- **битовый массив** B размера m, изначально весь нулевой — это и есть всё «хранилище» фильтра;
- k **независимых хеш-функций** $h_1, h_2, \ldots, h_k$. Каждая принимает ключ x и возвращает **позицию** (целое число) в диапазоне $[0, m)$ — то есть индекс в массиве B.

Самих ключей, ссылок на SSTable, счётчиков — ничего этого фильтр не хранит. Всё его «знание» о множестве закодировано в том, какие биты в B стоят в единицу.

На что именно возвращает хеш-функция. Обычно берут быструю 64-битную хеш-функцию (MurmurHash3, xxHash) и сводят её выход в нужный диапазон:

$$h_i(x) = \mathrm{hash}_i(x) \bmod m$$

или, если m это степень двойки, через дешёвую маску:

$$h_i(x) = \mathrm{hash}_i(x) \ (m - 1)$$

То есть одна хеш-функция $\longrightarrow$ одна позиция $\longrightarrow$ один бит в B, который она «задевает».

Вставка `Add(x)`

Алгоритм буквально из трёх шагов:

1. посчитать k позиций $p_1 = h_1(x), \ldots, p_k = h_k(x)$;
2. выставить `B[p_i] = 1` для каждого i;
3. готово — ключ x «добавлен».

Заметим: если какой-то из этих битов уже был 1 (его раньше выставил другой ключ), мы просто оставляем его 1. Фильтр никак не различает «этот бит мой» и «этот бит выставил кто-то ещё» — именно отсюда в дальнейшем возьмутся ложные срабатывания.

Проверка `MightContain(x)`

Тоже три шага:

1. посчитать те же k позиций $p_1, ..., p_k$ (используя ровно те же хеш-функции, что и при вставке);
2. прочитать биты `B[p_1], ..., B[p_k]`;
3. вернуть ответ по правилу:
   - если **хотя бы один** из этих битов равен 0 ⟶ ключа **точно нет** в множестве;
   - если **все** k битов равны 1 ⟶ ключ **возможно** есть (может быть ложным срабатыванием).

Маленький пример на пальцах

Возьмём $m = 16$ (то есть битовый массив длины 16) и $k = 3$ хеш-функции. Начальное состояние:

```
index:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
B:      0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
```

Добавляем ключ `"apple"`. Пусть хеши вернули позиции $h_1 = 2,\ \ h_2 = 7,\ \ h_3 = 13$. Выставляем биты 2, 7, 13:

```
B:      0  0  1  0  0  0  0  1  0  0  0  0  0  1  0  0
```

Добавляем ключ `"banana"`. Пусть хеши дали $h_1 = 4$, $h_2 = 7$, $h_3 = 11$. Позиция 7 уже была 1 — и это нормально:

```
B:      0  0  1  0  1  0  0  1  0  0  0  1  0  1  0  0
```

Теперь проверяем:

- `MightContain("apple")` ⟶ смотрим позиции 2, 7, 13 ⟶ `1, 1, 1` ⟶ «возможно, есть». И это правда, мы `apple` добавляли.
- `MightContain("cherry")` ⟶ пусть хеши дали 1, 5, 9 ⟶ смотрим `B[1]=0` ⟶ **точно нет**. Останавливаемся на первом же нуле, дальше можно не проверять.
- `MightContain("grape")` ⟶ пусть хеши дали 2, 4, 11 ⟶ смотрим `1, 1, 1` ⟶ «возможно, есть». Но `grape` мы никогда не добавляли — это классическое **ложное срабатывание**: все нужные биты случайно выставили другие ключи (`apple` поставил бит 2, `banana` поставила биты 4 и 11).

Почему нет ложных отрицаний

Если ключ x был добавлен, то `Add(x)` точно выставил `B[h_1(x)] = ... = B[h_k(x)] = 1`. Никто эти биты обратно в 0 не снимает (фильтр иммутабельный, как и SSTable над которой он построен). Значит позже `MightContain(x)` увидит все k битов единицами и вернёт «возможно, есть». То есть «забыть» уже добавленный ключ фильтр не может.

Почему бывают ложные положительные

`MightContain(y)` для незнакомого ключа y вернёт «возможно, есть», если все k его битов случайно оказались выставленными другими ключами (как в примере с `grape`). Чем больше ключей мы уже запихнули в фильтр, тем больше в B единиц, и тем выше шанс такого совпадения. Дальше в разделе про математику мы оценим эту вероятность количественно.

Немного математики

Пусть в фильтре уже n ключей, массив имеет m бит и используется k хеш-функций. Вероятность, что конкретный бит остался нулевым после всех вставок:

$$\left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$$

Тогда вероятность ложного срабатывания (все k бит для нового ключа оказались единицами):

$$p \approx \left(1 - e^{-kn/m}\right)^k$$

Оптимальное число хеш-функций при фиксированных m и n:

$$k^* = \frac{m}{n} \ln 2 \approx 0,693 \cdot \frac{m}{n}$$

При этом выбор m под заданную ошибку p в пересчёте на один ключ:

$$\frac{m}{n} = -\frac{\ln p}{(\ln 2)^2} \approx -1,44 \cdot \log_2 p$$

Практические ориентиры (битов на ключ):

Целевой false positive	m/n	k^*
10%	$\approx 4,8$	3
1%	$\approx 9,6$	7
$0,1\%$	$\approx 14,4$	10
$0,01\%$	$\approx 19,2$	13

Типичная настройка в RocksDB и LevelDB — `10` бит на ключ, что даёт примерно 1% ложных срабатываний при $k = 7$.

Сколько это памяти на практике: SSTable с 10^6 ключей при 10 бит на ключ — это $\approx 1,25$ МБ bloom-фильтра. Для базы в 10^9 записей суммарный размер фильтров $\approx 1,25$ ГБ, и такой фильтр удобно держать в RAM.

Место в чтении LSM

Когда приходит `Get(key)`, LSM идёт по SSTable сверху вниз (от свежих уровней к старым). Перед тем как лезть в индекс и блоки данных конкретной SSTable:

1. считаются k хешей от `key`;
2. проверяются k битов в bloom-фильтре, который уже лежит в памяти;
3. если хоть один 0 — SSTable пропускается без единого дискового обращения;
4. если все 1 — делается обычное чтение индекса и блока, и это может быть ложная тревога.

Особенно сильно фильтр помогает на запросах по несуществующим ключам (типично в кэш-сценариях: «есть ли сессия с таким id?»). Без фильтра такой запрос спускается по всем уровням и делает L лишних чтений.

Тонкие места и ограничения

- **Нет удаления отдельных ключей.** Обычный bloom-фильтр не умеет «забывать». В LSM это не проблема: каждая SSTable иммутабельна, а при компакции создаётся новая SSTable и новый фильтр.
- **Нет поддержки range-запросов.** Фильтр отвечает только про точное равенство. Для диапазонных чтений используют отдельные структуры, например prefix Bloom или SuRF.
- **Независимость хешей.** На практике используют один быстрый хеш (MurmurHash, xxHash) и получают k функций через формулу $h_i(x) = h_a(x) + i \cdot h_b(x)$. Это корректно при больших m.
- **Стоимость ложного срабатывания.** Одно ложное срабатывание стоит одно «лишнее» чтение блока из SSTable. Поэтому 1% ложных срабатываний — это примерно 1% лишних чтений, что обычно терпимо.

Go-псевдоинтерфейс и оценки:

```go
// BloomFilter: вероятностная проверка наличия.
type BloomFilter interface {
    // Add: выставляет k битов, по одному на каждую хеш-функцию.
    // O(k).
    Add(key []byte)

    // MightContain: проверяет k битов.
    // Возвращает false → точно нет;
    // возвращает true  → возможно, есть (возможен ложный положительный ответ).
    // O(k).
    MightContain(key []byte) bool
}
```

WAL

WAL (Write-Ahead Log) это append-only журнал на диске. Каждая запись сначала фиксируется в WAL и только потом применяется к Memtable. При крахе процесса Memtable восстанавливается из WAL, так что подтверждённые записи не теряются.

Полная структура

LSM-дерево это конвейер: запись идёт быстро сверху вниз, а порядок поддерживается фоновыми слияниями.

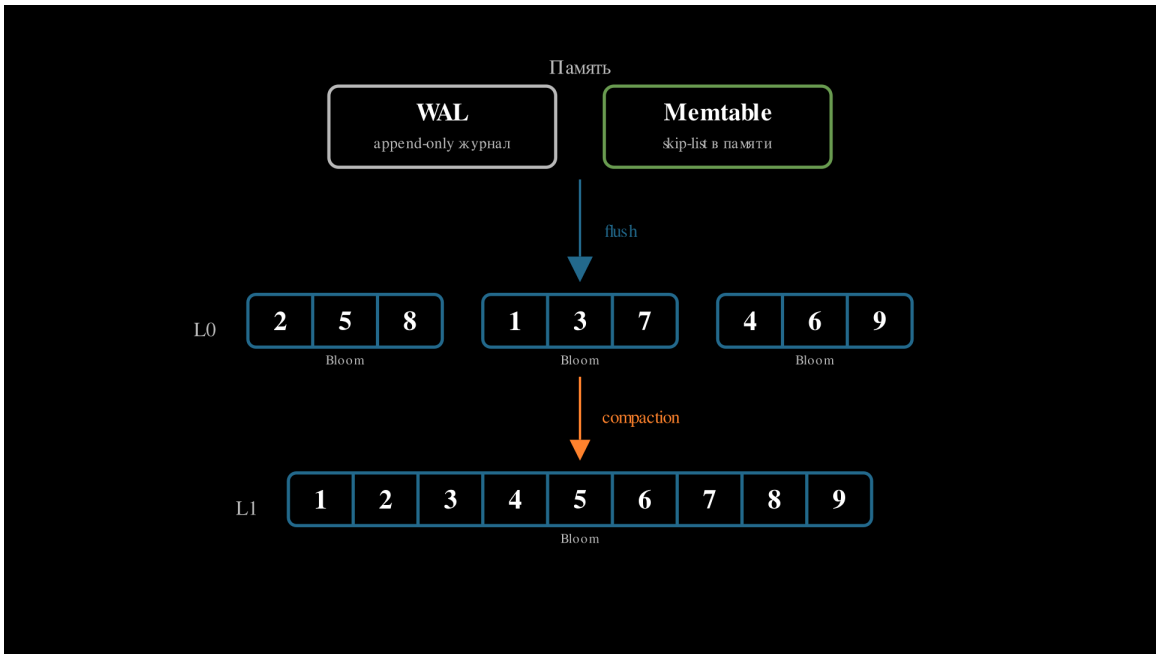

Figure 1: Пайплайн LSM: Memtable в памяти, несколько SSTable на L0, слитый SSTable на L1.

Основные части:

1. WAL — журнал для надёжности, каждая запись сначала фиксируется последовательно.
2. Memtable — быстрая запись и сортировка в памяти.
3. Immutable Memtable — замороженная память, ожидающая сброса.
4. Набор SSTable на диске — отсортированные файлы, каждый с Bloom Filter.

Как работает запись:

- клиент пишет `(ключ, значение)`;
- запись попадает в WAL и Memtable (быстро, в памяти);
- при переполнении Memtable замораживается и сбрасывается в новый SSTable (последовательный I/O).

Как работает чтение:

- поиск идёт от самых свежих структур к старым: Memtable ⟶ Immutable Memtable ⟶ SSTable L0 ⟶ SSTable L1, ...;

- перед чтением SSTable проверяется Bloom Filter, чтобы не читать лишнее с диска;
- результат берётся из первой найденной версии.

Как поддерживается порядок и место:

- на диске постепенно накапливаются SSTable с пересекающимися ключами;
- фоновая компакция сливает их в более крупные и упорядоченные файлы;
- при слиянии старые версии и tombstone-записи выкидываются, освобождая место.

Пайплайн в движении

Анимация показывает две последовательные Memtable-пачки, которые сбрасываются в два SSTable на L0, после чего компакция сливает их в один отсортированный SSTable на L1.

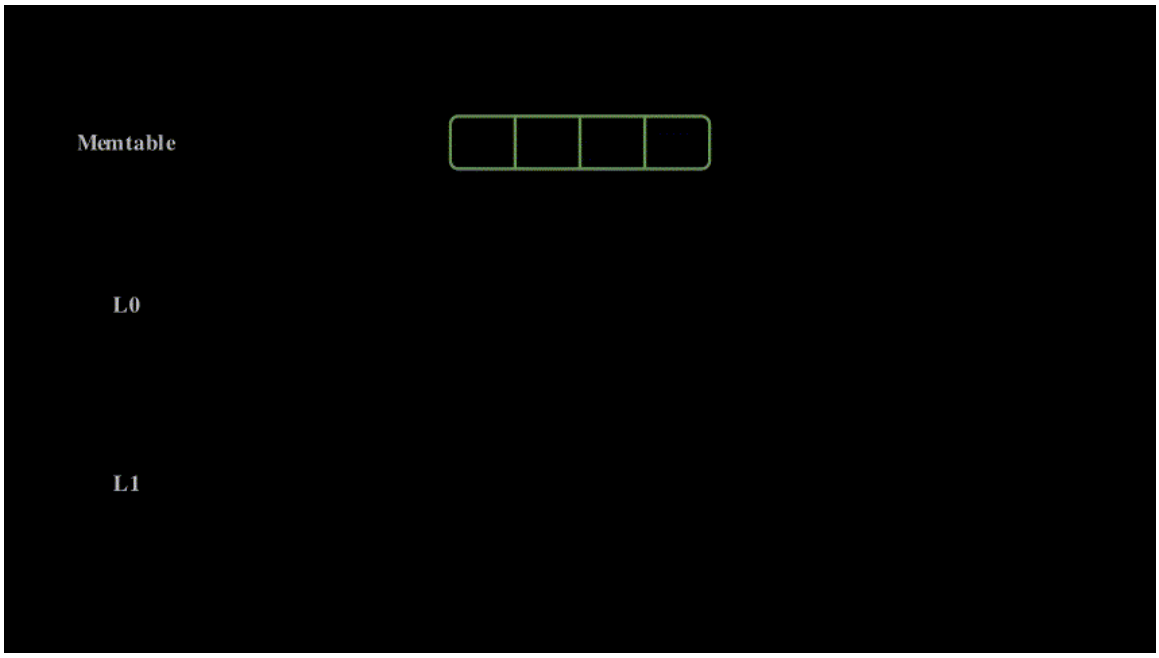

Figure 2: Конвейер LSM: вставки в Memtable, flush на L0, компакция на L1.

Сложность операций

Create / Update

$$O(\log m)$$

- запись в WAL: $O(1)$ последовательная;
- вставка в Memtable: $O(\log m)$, где m это число ключей в Memtable;
- periodic flush: $O(m)$ последовательной записи, амортизируется по всем вставкам.

Read

$$O(\log m + L \cdot \log b)$$

- поиск в Memtable: $O(\log m)$;
- по каждому уровню из L Bloom Filter отсекает ненужные SSTable;

- в оставшемся SSTable бинарный поиск по индексу блоков: $O(\log b)$.

Delete

$$O(\log m)$$

- удаление это вставка специальной tombstone-записи в Memtable;
- физическое удаление случится во время ближайшей компакции, которая пройдёт по соответствующему уровню.

Range Query

$$O(\log m + R)$$

- находим левые границы во всех актуальных структурах;
- сливаем отсортированные потоки на лету, получая R записей в результате.

Сравнение с B-tree и Hash

LSM vs B-tree

- **Чтение:** B-tree за $O(\log_B n)$ попадает в нужный лист; LSM может обойти Memtable и несколько SSTable (Bloom Filter ускоряет, но не гарантирует).
- **Запись:** LSM обычно быстрее — append-only в Memtable и последовательный flush, без read-modify-write узлов.
- **Стабильность:** B-tree даёт предсказуемую задержку; у LSM бывают всплески из-за компакции.
- **Итого:** B-tree для OLTP и low-latency чтений; LSM для write-heavy и time-series.

LSM vs Hash

- hash даёт $O(1)$ по ключу, но теряет сортировку и не поддерживает range-сканы;
- LSM хорош для диапазонных запросов и потоков событий.

Практическое применение

Хорошо подходит:

- частые вставки и обновления по ключу (write-heavy нагрузки);
- пакетные записи и стриминг событий;
- диапазонные запросы по ключу;
- чтение свежих данных, где важна скорость записи и допустима амортизированная задержка.

Плохо подходит:

- случайные чтения по множеству ключей без кэша (много SSTable);
- низкая и стабильная задержка чтения в p99 (компакция и пересечения);
- частые операции удаления, если критично немедленно освободить место.

Типичные системы на этой идее:

Система	Зачем LSM
RocksDB / LevelDB	базовый KV-движок от Google/Meta, встраивается в десятки систем
Cassandra	write-heavy нагрузки, time-series, распределённые логи
HBase	Hadoop-экосистема, аналитика поверх HDFS
Google Bigtable	семинальная система, откуда LSM пошёл в prod
InfluxDB	time-series: постоянный поток метрик, редкие обновления

B-дерево: индексирование на диске

LSM-дерево оптимизировано под запись, но если нам нужна быстрая и предсказуемая выборка по ключу, обычно приходят к **B-дереву**. Это классическая дисковая структура, на которой построены индексы в PostgreSQL, MySQL, Oracle и SQLite.

Главная идея этой лекции: **B-дерево хранит на диске изменяемые страницы**, а не набор иммутабельных файлов. Именно из этого следуют и его сильные стороны на чтении, и его цена на записи.

Почему для диска используют B-дерево

Ключевая идея B-дерева проста: хранить ключи не по одному, а группами в страницах размером с дисковый блок, обычно `4-16 KB`. Одна страница хранит сразу много ключей, поэтому коэффициент ветвления получается большим, а высота дерева маленькой. Для поиска среди миллиарда записей часто достаточно примерно `3-4` чтений с диска:

$$\log_{1000}(10^9) \approx 3$$

Компромисс тоже понятен: B-дерево хорошо работает на чтении, но пишет медленнее, чем LSM-дерево. Любая модификация обычно требует прочитать страницу, изменить её содержимое и записать обратно. Такая схема напрямую связана со стоимостью операций ввода-вывода.

Изменяемые страницы против иммутабельных файлов

На этом уровне B-дерево и LSM принципиально различаются:

- в LSM на диск попадают **иммутабельные SSTable**; запись идёт через append и последующую compaction;
- в B-дереве на диске лежат **изменяемые страницы**; вставка и удаление меняют уже существующие узлы;

- поэтому у B-дерева чтение обычно короче и стабильнее, а запись чаще превращается в `read-modify-write` по нескольким страницам.

Это и есть важная связь между перестройкой структуры и I/O:

- если целевой лист ещё не заполнен, меняется небольшое число страниц на пути от корня к листу;
- если лист переполнен, происходит split, и нужно записать уже не одну, а несколько страниц;
- если split поднимается выше, дорожает и запись, и объём случайного I/O;
- при этом база не пересобирает всё дерево целиком, а локально реорганизует только затронутый путь.

Структура и коэффициент ветвления

Коэффициент ветвления это максимальное число потомков внутреннего узла. Тогда сам узел хранит не более `B-1` ключей.

Структура узла:

- ключи: $[k_1, k_2, \ldots, k_{B-1}]$ в отсортированном порядке;
- ссылки на дочерние узлы: $[p_0, p_1, \ldots, p_B]$;
- поддерево по ссылке p_0 хранит ключи `< k_1`;
- поддерево по ссылке p_i для `0 < i < m` хранит ключи между k_i и k_{i+1};
- поддерево по ссылке p_m хранит ключи `> k_m`, где m это число ключей в узле;
- листовой узел содержит значения вместо указателей на поддеревья;
- каждый узел обычно соответствует одному дисковому обращению.

Выбор `B` связан с размером блока и размером внутренней записи. Если страница равна `8 KB`, а одна внутренняя запись вместе с указателем, служебными байтами и выравниванием занимает порядка `24-32` байт, то коэффициент ветвления уже получается порядка нескольких сотен. Этого достаточно, чтобы даже на очень больших данных глубина дерева оставалась маленькой.

В реальных системах коэффициент ветвления обычно измеряется не единицами, а десятками и сотнями. Это естественное следствие того, что одна страница содержит много ключей и ссылок:

- в PostgreSQL страница обычно имеет размер `8 KB`;
- в SQLite по умолчанию часто используется `4 KB`;
- в InnoDB по умолчанию типичен размер страницы `16 KB`.

Точное значение зависит от длины ключа, служебных полей и степени заполнения страницы, поэтому здесь важны именно оценки. Если внутренняя запись узла занимает порядка `24-32` байт, то грубая верхняя оценка по размеру страницы получается такой:

Система	Типичный размер страницы	Грубая оценка `B`
SQLite	`4 KB`	примерно `100-170` потомков

Система	Типичный размер страницы	Грубая оценка `B`
PostgreSQL	`8 KB`	примерно `200-340` потомков
InnoDB	`16 KB`	примерно `400-680` потомков

Это именно оценка по I/O-модели, а не жёсткая константа движка. В реальном индексе часть места уйдёт на заголовки страниц, разделители, выравнивание и неполное заполнение. Но вывод не меняется: большой `B` резко уменьшает высоту дерева и тем самым сокращает число дисковых обращений.

Инварианты

- все листья находятся на одной глубине;
- внутренний узел содержит от $\lceil B/2 \rceil$ до $B - 1$ ключей;
- корень содержит минимум один ключ.

Пример узла при `B = 4`

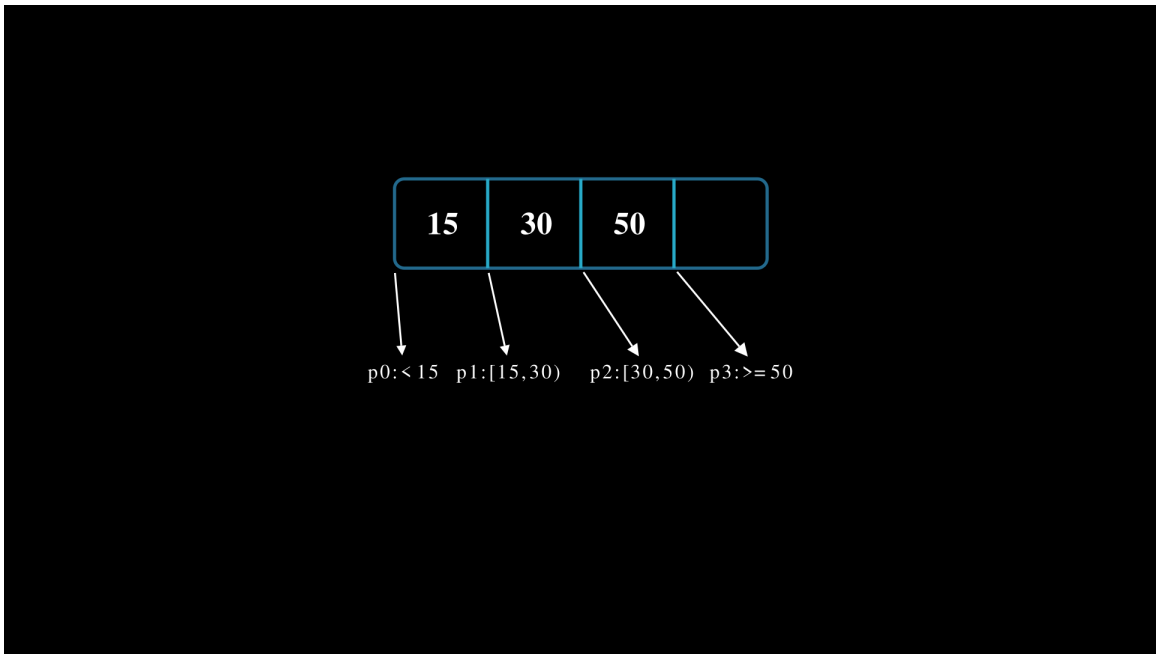

Figure 3: Схема узла B-дерева при `B = 4`, собранная из manim-сцены.

Вставка и балансировка

Пример роста B-дерева в учебной конфигурации с максимумом `3` ключей в узле при вставке `8, 21, 34, 55, 63, 77, 84, 96`:

Figure 4: Анимация роста B-дерева при последовательной вставке и расщеплении корня, собранная из manim-сцены.

При переполнении узла происходит следующее:

- узел расщепляется на две части;
- медианный ключ поднимается в родителя;
- если родитель тоже переполнен, расщепление распространяется вверх;
- корень может вырасти, и тогда высота дерева увеличивается на единицу.

Важно не перепутать локальную перестройку с полной пересборкой индекса. При обычной вставке база данных не перестраивает всё дерево заново. Она изменяет лишь несколько страниц на пути от корня к нужному листу. Если листовая страница переполнена, происходит её расщепление; затем может потребоваться обновить родительскую страницу, а иногда и несколько страниц выше.

Из-за этого расщепление действительно увеличивает число операций ввода-вывода:

- нужно прочитать путь от корня к листу;
- нужно изменить переполненную страницу;
- нужно записать новую страницу, появившуюся после расщепления;
- нужно обновить родительскую страницу;
- в редком случае изменения поднимаются до корня.

Поэтому на анимации такая операция выглядит как дорогая перестройка. Но в реальной базе данных это обычно не полная перестройка индекса, а локальная реорганизация нескольких страниц. Полная перестройка индекса встречается реже и стоит значительно дороже, потому что требует заново построить всю структуру по данным таблицы.

Сложность операций

Вставка

$$O(\log_B n)$$

- поиск листового узла: $O(\log_B n)$ дисковых обращений;
- вставка в лист: $O(1)$;
- при переполнении возможны расщепления вплоть до корня.

Поиск

$$O(\log_B n)$$

- внутри узла поиск идёт в памяти;
- переход к дочернему узлу стоит одно дисковое обращение;
- максимум получается $O(\log_B n)$ обращений.

Обновление

$$O(\log_B n)$$

- сначала находится нужный ключ;
- затем значение меняется на месте.

Удаление

$$O(\log_B n)$$

- удаляем ключ;
- если ключей стало слишком мало, нужно слияние с соседним узлом или перераспределение записей.

Диапазонный запрос

$$O(\log_B n + R/B)$$

где R это размер результата.

- сначала ищем левую границу;
- затем последовательно сканируем листья, используя локальность хранения.

Сравнение с LSM-деревом по I/O

B-дерево и LSM-дерево

- **Чтение:** у B-дерева это $O(\log_B n)$ обращений к диску. У LSM-дерева нужно проверить оперативную таблицу и несколько неизменяемых файлов.
- **Запись:** LSM-дерево обычно быстрее, потому что пишет данные последовательно и избегает постоянного чтения и переписывания одних и тех же страниц.
- **Стабильность задержек:** B-дерево даёт более предсказуемое время ответа; у LSM-дерева возможны всплески из-за крупных слияний.

- **Итог:** B-дерево хорошо подходит для транзакционных систем с жёсткими требованиями к задержке чтения, а LSM-дерево часто выигрывает в сценариях с преобладанием записи.

Практическое применение

Хорошо подходит:

- случайные чтения по ключу с жёсткими требованиями к задержке;
- смешанные нагрузки на чтение и запись;
- реляционные БД и файловые системы.

Плохо подходит:

- сценарии с преобладанием записи;
- потоки последовательных вставок, где важнее пропускная способность;
- журнальные потоки событий, где лучше подходят LSM-структуры.

Индексы: B-tree, hash и bitmap

После лекций про LSM и B-дерево полезно отделить два уровня разговора:

- **структура хранения и цена I/O**: immutable SSTable против mutable страниц;
- **тип индекса как способ доступа**: B-tree, hash, bitmap и их пригодность под разные запросы.

Эта лекция про второй уровень: что вообще делает индекс и почему у разных индексов разные сильные стороны.

Что такое индекс

Индекс в базе данных это отдельная структура, которая помогает найти нужные строки без полного сканирования таблицы.

Без индекса запрос вида

```
select * from users where email = 'a@b.c';
```

часто означает:

1. прочитать много страниц таблицы;
2. проверить условие для каждой строки;
3. выбрать подходящие строки.

С индексом база сначала идёт в компактную вспомогательную структуру, находит нужный ключ, а уже потом переходит к данным. Обычно индекс хранит что-то вроде:

$$\text{search key} \rightarrow \text{row id} \ / \ \text{page id} \ / \ \text{primary key}$$

То есть индекс это не “таблица поменьше”, а **дополнительный путь доступа** к данным.

Почему индекс ускоряет чтение, но замедляет запись

Индекс экономит I/O на чтении, если запрос достаточно селективен: вместо чтения многих страниц таблицы можно прочитать несколько страниц индекса и затем перейти только к нужным строкам.

Но за это приходится платить:

- при `insert` надо добавить запись и в таблицу, и в индекс;
- при `update` индекс может потребовать перестройки, если изменился индексируемый столбец;
- при `delete` запись нужно убрать или пометить удалённой и в таблице, и в индексе;
- чем больше индексов на таблице, тем дороже каждая запись.

Поэтому индекс это всегда обмен: **ускоряем часть чтений ценой памяти, диска и более дорогих записей**.

B-tree как индекс по умолчанию

B-tree это самый универсальный индекс. Он хранит ключи в отсортированном порядке и поэтому умеет не только точечный поиск, но и работу с порядком ключей.

Что B-tree делает хорошо:

- поиск по равенству: `=`, `IN`;
- диапазоны: `<`, `<=`, `>`, `between`;
- префиксные условия;
- `order by`, `min`, `max`;
- предсказуемое поведение на смешанной OLTP-нагрузке.

Почему он универсален:

- ключи упорядочены;
- высота дерева мала из-за большого branching factor;
- чтение сводится к короткому пути по страницам;
- цена записи понятна: локальные split/merge страниц.

Если нет специальной причины выбрать что-то ещё, на практике чаще всего начинают именно с B-tree.

Hash-индекс

Hash-индекс пропускает ключ через хеш-функцию и кладёт его в корзину:

$$h(k) \to \text{bucket}$$

Сильная сторона hash-индекса:

- очень быстрый поиск **по точному равенству**;
- хорош, когда запросы почти всегда выглядят как `where id = ...`.

Ограничения:

- он не хранит ключи в сортированном порядке;
- не умеет диапазоны;
- не помогает для `order by`;
- плохо подходит для префиксного поиска.

Именно поэтому hash-индекс уже, чем B-tree. Он может быть быстрее в своей узкой задаче, но как индекс “по умолчанию” почти всегда проигрывает B-tree по универсальности.

Bitmap-индекс

Bitmap-индекс особенно полезен там, где столбец принимает **не очень много разных значений**. Для каждого значения хранится битовая маска: в каком наборе строк это значение встречается.

Пример для столбца `gender`:

- bitmap для `M`;
- bitmap для `F`.

Для столбца `status`:

- bitmap для `new`;
- bitmap для `paid`;
- bitmap для `cancelled`.

Почему это мощно в аналитике:

- фильтры можно быстро объединять через `AND`, `OR`, `NOT`;
- получается очень дешёвая работа с несколькими низкокардинальными предикатами;
- хорошо сжимается и хорошо подходит под DWH-нагрузки.

Почему bitmap плох в OLTP:

- частые обновления заставляют постоянно править битовые векторы;
- при высокой кардинальности bitmap разрастается и теряет смысл;
- индекс менее удобен для точечных обновлений и горячих записей.

Проще всего запомнить так: **bitmap любит аналитику и редкие изменения, B-tree любит универсальность, hash любит точное равенство**.

Как выбирать индекс

Тип запроса или нагрузки	Что обычно выбирать
`id = x`, точечный поиск	`B-tree`, иногда `hash`
`created_at between ...`, диапазоны	`B-tree`
`order by`, `min`, `max`	`B-tree`
несколько фильтров по низкокардинальным полям в аналитике	`bitmap`

Тип запроса или нагрузки	Что обычно выбирать
много записей, мало обновлений, OLAP/DWH	`bitmap` часто уместен
смешанная транзакционная нагрузка	`B-tree`

Связь с двумя предыдущими лекциями

Теперь различие можно формулировать чище:

- **LSM** и **B-tree** это прежде всего разговор о том, как структура живёт на диске и как она платит за I/O;
- **B-tree, hash и bitmap** это разговор о типах индексов и о том, какие запросы они ускоряют;
- при этом B-tree одновременно попадает в обе категории: это и дисковая структура, и самый распространённый тип индекса.

Именно поэтому логично держать отдельные лекции:

1. `LSM-дерево`: immutable файлы, flush, compaction, write amplification.
2. `B-дерево`: mutable страницы, branching factor, split/merge, чтение с диска.
3. `Индексы`: что такое индекс вообще, и когда выбирать B-tree, hash или bitmap.